ALU

Angel Alam Mateo Encarnación
Mecatrónica
ITLA
La Caleta, Santo Domingo
Cristian Miguel Jimenez
20240712@itla.edu.do

Resumen: ***En este laboratorio se llevarán a cabo distintas operaciones vinculadas a la unidad lógica aritmética, con el propósito de analizar sus aplicaciones tanto en el ámbito práctico como en el teórico, siendo este un componente fundamental en la electrónica digital.***

Abstract: ***During this lab, various functions related to the arithmetic logic unit will be performed in order to see both the practical and theoretical applications of this key component in the world of digital electronics.***

Palabras claves: ALU.

I. Introducción

Durante el siguiente documento se estará diseñando un ALU mediante lógica combinacional con el fin de poder realizar operaciones lógicas y aritméticas de cuatro bits.

II. Marco Teórico

ALU: Una Unidad Aritmético-Lógica (ALU) es un elemento esencial dentro de los procesadores, encargado de ejecutar operaciones aritméticas, como suma y resta, así como operaciones lógicas, como AND, OR y XOR, entre otras, utilizando datos binarios. Su funcionamiento se basa en recibir dos operandos de entrada junto con una señal de control que define la operación a realizar, generando como salida el resultado y ciertas señales de estado, como acarreo, desbordamiento o condición de cero. Este componente es clave en el procesamiento de información en CPUs, microcontroladores y sistemas digitales, y puede implementarse de manera sencilla mediante lenguajes como Verilog en proyectos con FPGA o en entornos de simulación digital.

III. Diseño del circuito

Para el diseño de la ALU, se partirá de las funciones asignadas, las cuales serán seleccionadas en parte por elección propia y en parte según la matrícula. Estas funciones se escogerán a partir de la siguiente tabla:

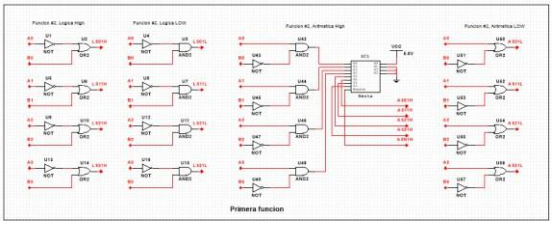
	MODE SELECT INPUTS				ACTIVE LOW INPUTS &OUTPUTS		ACTIVE HIGH INPUTS &OUTPUTS	
	S3	S2	S1	S0	LOGIC (M=H)	ARITHMETIC (M=L) (C_n=L)	LOGIC (M=H)	ARITHMETIC (M=L) (C_n=H)
0	L	L	L	L	$\overline{A}$	A minus 1	$\overline{A}$	A
1	L	L	L	H	$\overline{AB}$	AB minus 1	$\overline{A+B}$	A+B
2	L	L	H	L	$\overline{A}+B$	$A\overline{B}$ minus 1	$\overline{AB}$	$A+\overline{B}$
3	L	L	H	H	Logical 1	minus 1	Logical 0	Minus 1
4	L	H	L	H	$\overline{B}$	AB plus $(A+\overline{B})$	$\overline{B}$	$(A+B)$ plus $A\overline{B}$
5	L	H	L	L	$\overline{A+B}$	A plus $(A+\overline{B})$	$\overline{AB}$	A plus $A\overline{B}$
6	L	H	H	L	$\overline{A \oplus B}$	A minus B minus 1	$A \oplus B$	A minus B minus 1
7	L	H	H	H	$A+\overline{B}$	$A+\overline{B}$	$A\overline{B}$	$A\overline{B}$ minus 1
8	H	L	L	L	$\overline{AB}$	A plus (A+B)	$\overline{A}+B$	A plus AB
9	H	L	L	H	$A \oplus B$	A plus B	$\overline{A \oplus B}$	A plus B
10	H	L	H	L	B	$A\overline{B}$ plus (A+B)	B	$(A+\overline{B})$ plus AB
11	H	L	H	H	A+B	A+B	AB	AB minus 1
12	H	H	L	L	Logical 0	A plus A	Logical 1	A plus A
13	H	H	L	H	$A\overline{B}$	AB plus A	$A+\overline{B}$	$(A+B)$ plus A
14	H	H	H	L	AB	$A\overline{B}$ plus A	A+B	$(A+\overline{B})$ plus A
15	H	H	H	H	A	A	A	A minus 1

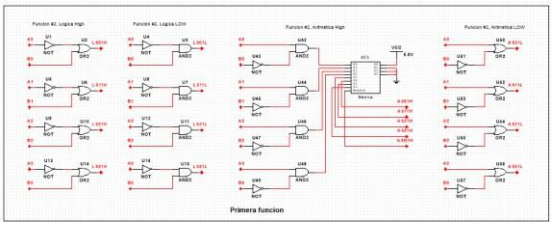
Y las funciones elegidas son las siguientes:

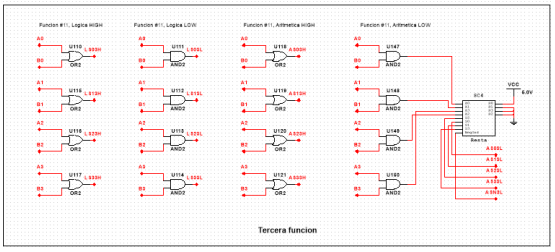
Tabla de verdad							
Funcion:	A	B	C	Logic H	Arit H	Logic L	Arit L
2	0	0	0	A'+B	AB' minus 1	A'B	A+B'
9	0	0	1	A⊕B	A plus B	A'⊕B'	A plus B
11	0	1	0	A+B	A+B	AB	AB minus 1
13	0	1	1	AB'	AB plus A	A+B'	(A+B) plus A
8	1	0	0	A'B	A plus (A+B)	A'+B	A plus AB
4	1	0	1	B'	AB plus (A+B')	B'	(A+B) plus AB'
OFF	1	1	0	X	X	X	X
OFF	1	1	1	X	X	X	X

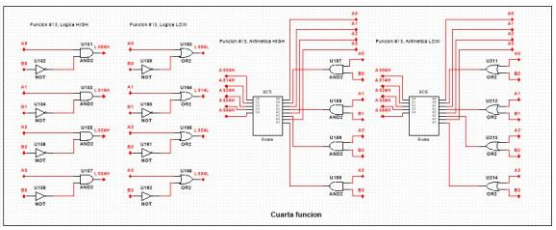
Partiendo de esta tabla, podemos construir los circuitos logicos en multsim:

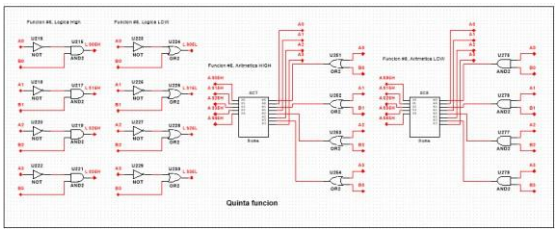
Funciones en Multisim:

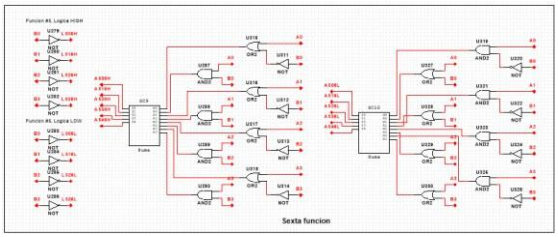




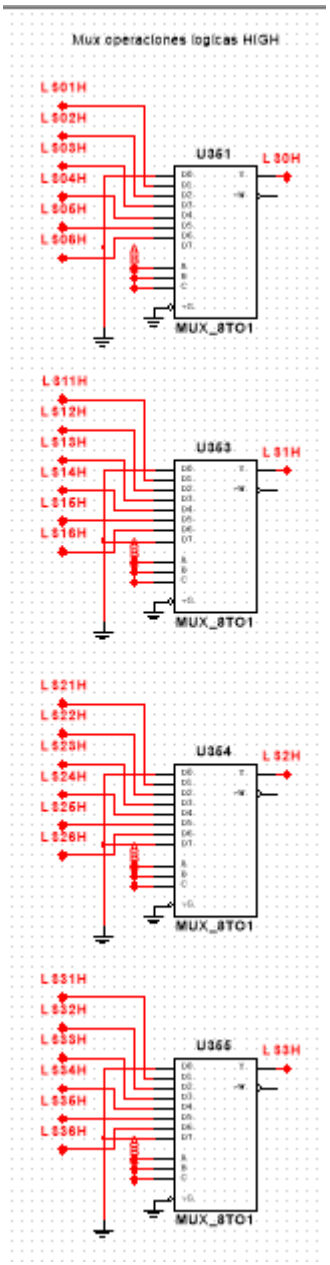


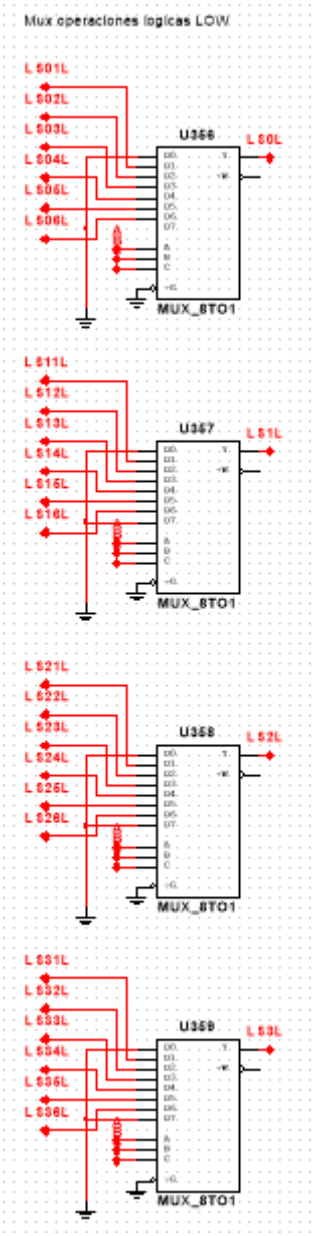


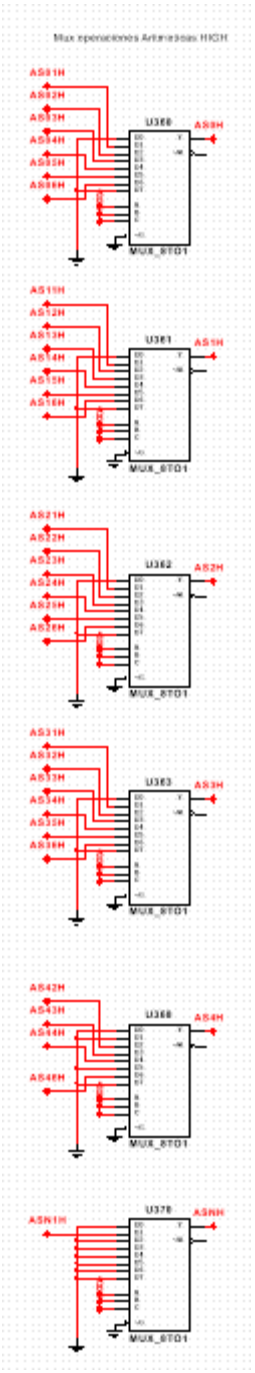


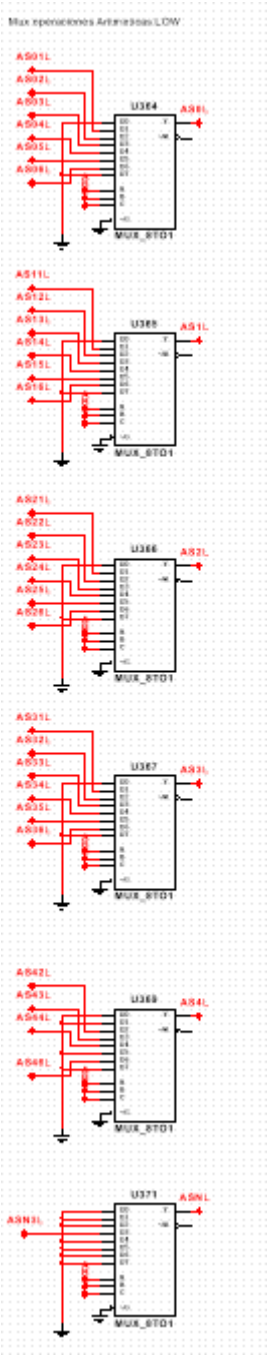


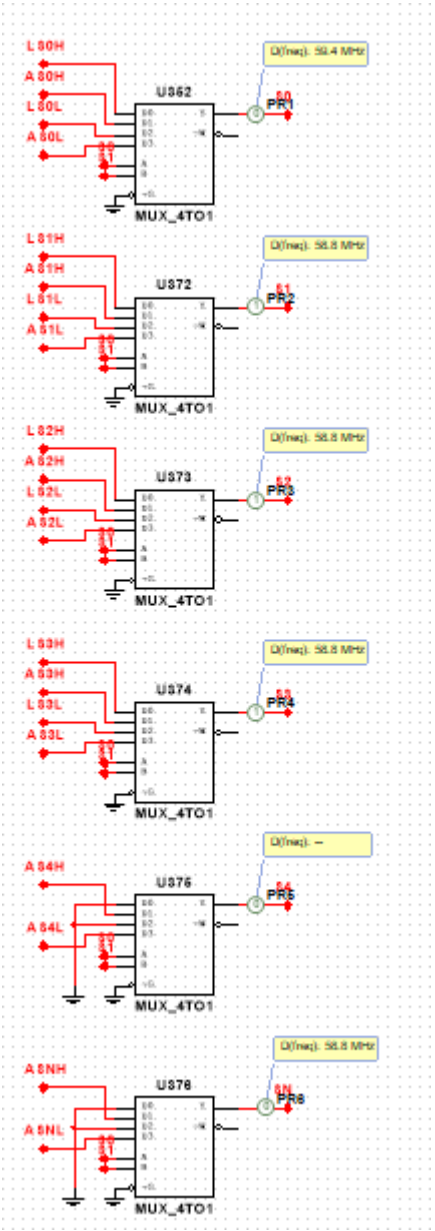
Una vez construida las funciones logicas, podemos construir los multiplexores, los cuales se encargaran de monitorear cual funcion es visible:




Mux operaciones logicas LOW
L S01L
L S02L
L S03L
L S04L
L S05L
L S06L
U356
L S0L
MUX_8TO1
L S11L
L S12L
L S13L
L S14L
L S15L
L S16L
U357
L S1L
MUX_8TO1
L S21L
L S22L
L S23L
L S24L
L S25L
L S26L
U358
L S2L
MUX_8TO1
L S31L
L S32L
L S33L
L S34L
L S35L
L S36L
U359
L S3L
MUX_8TO1


Mux operaciones Aritmeticas HIGH
AS01H
AS02H
AS03H
AS04H
AS05H
AS06H
U360
AS0H
MUX_8TO1
AS11H
AS12H
AS13H
AS14H
AS15H
AS16H
U361
AS1H
MUX_8TO1
AS21H
AS22H
AS23H
AS24H
AS25H
AS26H
U362
AS2H
MUX_8TO1
AS31H
AS32H
AS33H
AS34H
AS35H
AS36H
U363
AS3H
MUX_8TO1
AS42H
AS43H
AS44H
AS46H
U368
AS4H
MUX_8TO1
ASN1H
U378
ASNH
MUX_8TO1





Para concluir, llevaremos a verilog las funciones creadas y haremos la implementacion en fisico:

```
module operation_block_2 (
    input wire A0, A1, A2, A3, B0, B1, B2, B3,
    output wire OUT_AR1H, OUT_AR2H, OUT_AR3H, OUT_AR4H, OUT_ARNH,
    output reg OUT_AR1L, OUT_AR2L, OUT_AR3L, OUT_AR4L,
    output reg OUT_LG1H, OUT_LG2H, OUT_LG3H, OUT_LG4H,
    output reg OUT_LG1L, OUT_LG2L, OUT_LG3L, OUT_LG4L
);

// Operaciones lógicas alto
always @(*) begin
    OUT_LG1H = ~A0 | B0;
    OUT_LG2H = ~A1 | B1;
    OUT_LG3H = ~A2 | B2;
    OUT_LG4H = ~A3 | B3;
end

// Operaciones lógicas bajo
always @(*) begin
    OUT_LG1L = ~A0 & B0;
    OUT_LG2L = ~A1 & B1;
    OUT_LG3L = ~A2 & B2;
    OUT_LG4L = ~A3 & B3;
end

// Preparación aritmética alto
reg x0, x1, x2, x3;
always @(*) begin
    x0 = A0 & ~B0;
    x1 = A1 & ~B1;
    x2 = A2 & ~B2;
    x3 = A3 & ~B3;
end

restadora INST_SUB (
    .A3(x3), .A2(x2), .A1(x1), .A0(x0),
    .B3(1'b0), .B2(1'b0), .B1(1'b0), .B0(1'b1),
    .S3(OUT_AR4H), .S2(OUT_AR3H), .S1(OUT_AR2H), .S0(OUT_AR1H), .C_OUT(OUT_ARNH)
);
```

Funcion #2

```
    output wire OUT_AR1H, OUT_AR2H, OUT_AR3H, OUT_AR4H, OUT_AR5H,
    output wire OUT_AR1L, OUT_AR2L, OUT_AR3L, OUT_AR4L, OUT_AR5L,
    output reg OUT_LG1H, OUT_LG2H, OUT_LG3H, OUT_LG4H,
    output reg OUT_LG1L, OUT_LG2L, OUT_LG3L, OUT_LG4L
);

// Inversión modo alto
always @(*) begin
    OUT_LG1H = ~B0;
    OUT_LG2H = ~B1;
    OUT_LG3H = ~B2;
    OUT_LG4H = ~B3;
end

// Inversión modo bajo
always @(*) begin
    OUT_LG1L = ~B0;
    OUT_LG2L = ~B1;
    OUT_LG3L = ~B2;
    OUT_LG4L = ~B3;
end

reg y0, y1, y2, y3;
reg y4, y5, y6, y7;
always @(*) begin
    y0 = A0 & B0;
    y1 = A1 & B1;
    y2 = A2 & B2;
    y3 = A3 & B3;
    y4 = A0 | ~B0;
    y5 = A1 | ~B1;
    y6 = A2 | ~B2;
    y7 = A3 | ~B3;
end

sumadora INST_ADD_H (
    .A3(y3), .A2(y2), .A1(y1), .A0(y0), .C_IN(1'b0),
    .B3(y7), .B2(y6), .B1(y5), .B0(y4),
```

Funcion #4

```
module operation_block_8 (
    input wire A0, A1, A2, A3, B0, B1, B2, B3,
    output wire OUT_AR1H, OUT_AR2H, OUT_AR3H, OUT_AR4H, OUT_AR5H,
    output wire OUT_AR1L, OUT_AR2L, OUT_AR3L, OUT_AR4L, OUT_AR5L,
    output reg OUT_LG1H, OUT_LG2H, OUT_LG3H, OUT_LG4H,
    output reg OUT_LG1L, OUT_LG2L, OUT_LG3L, OUT_LG4L
);

// Operaciones lógicas alto
always @(*) begin
    OUT_LG1H = ~A0 & B0;
    OUT_LG2H = ~A1 & B1;
    OUT_LG3H = ~A2 & B2;
    OUT_LG4H = ~A3 & B3;
end

// Operaciones lógicas bajo
always @(*) begin
    OUT_LG1L = ~A0 | B0;
    OUT_LG2L = ~A1 | B1;
    OUT_LG3L = ~A2 | B2;
    OUT_LG4L = ~A3 | B3;
end

// Datos intermedios alto
reg z0, z1, z2, z3;
always @(*) begin
    z0 = A0 | B0;
    z1 = A1 | B1;
    z2 = A2 | B2;
    z3 = A3 | B3;
end

sumadora INST_ADD_H (
    .A3(z3), .A2(z2), .A1(z1), .A0(z0), .C_IN(1'b0),
    .B3(A3), .B2(A2), .B1(A1), .B0(B0),
    .S3(OUT_AR4H), .S2(OUT_AR3H), .S1(OUT_AR2H), .S0(OUT_AR1H), .C_OUT(OUT_AR5H)
);
```

Funcion #8

```
module operation_block_9 (
    input wire A0, A1, A2, A3, B0, B1, B2, B3,
    output wire OUT_AR1H, OUT_AR2H, OUT_AR3H, OUT_AR4H, OUT_AR5H,
    output wire OUT_AR1L, OUT_AR2L, OUT_AR3L, OUT_AR4L, OUT_AR5L,
    output reg OUT_LG1H, OUT_LG2H, OUT_LG3H, OUT_LG4H,
    output reg OUT_LG1L, OUT_LG2L, OUT_LG3L, OUT_LG4L
);

// Lógica XOR modo alto
always @(*) begin
    OUT_LG1H = A0 ^ B0;
    OUT_LG2H = A1 ^ B1;
    OUT_LG3H = A2 ^ B2;
    OUT_LG4H = A3 ^ B3;
end

// Lógica XNOR modo bajo
always @(*) begin
    OUT_LG1L = A0 ~^ B0;
    OUT_LG2L = A1 ~^ B1;
    OUT_LG3L = A2 ~^ B2;
    OUT_LG4L = A3 ~^ B3;
end

// Suma aritmética alta
sumadora INST_ADD_H (
    .A3(A3), .A2(A2), .A1(A1), .A0(A0), .C_IN(1'b0),
    .B3(B3), .B2(B2), .B1(B1), .B0(B0),
    .S3(OUT_AR4H), .S2(OUT_AR3H), .S1(OUT_AR2H), .S0(OUT_AR1H), .C_OUT(OUT_AR5H)
);

// Suma aritmética baja
sumadora INST_ADD_L (
    .A3(A3), .A2(A2), .A1(A1), .A0(A0), .C_IN(1'b0),
    .B3(B3), .B2(B2), .B1(B1), .B0(B0),
    .S3(OUT_AR4L), .S2(OUT_AR3L), .S1(OUT_AR2L), .S0(OUT_AR1L), .C_OUT(OUT_AR5L)
);
endmodule
```

Funcion #9

```
module operation_block_11 (
    input wire A0, A1, A2, A3, B0, B1, B2, B3,
    output reg OUT_AR1H, OUT_AR2H, OUT_AR3H, OUT_AR4H,
    output wire OUT_AR1L, OUT_AR2L, OUT_AR3L, OUT_AR4L, OUT_AR5L,
    output reg OUT_LG1H, OUT_LG2H, OUT_LG3H, OUT_LG4H,
    output reg OUT_LG1L, OUT_LG2L, OUT_LG3L, OUT_LG4L
);

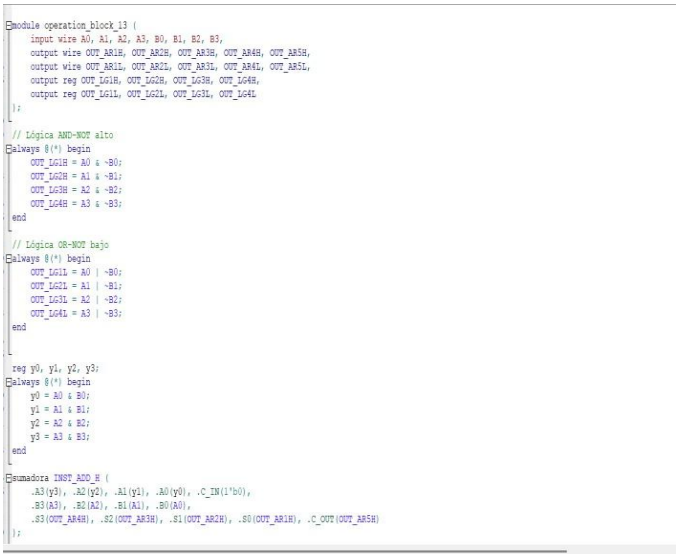
// Operador OR alto
always @(*) begin
    OUT_LG1H = A0 | B0;
    OUT_LG2H = A1 | B1;
    OUT_LG3H = A2 | B2;
    OUT_LG4H = A3 | B3;
end

// Operador AND bajo
always @(*) begin
    OUT_LG1L = A0 & B0;
    OUT_LG2L = A1 & B1;
    OUT_LG3L = A2 & B2;
    OUT_LG4L = A3 & B3;
end

// Aritmética OR alta
always @(*) begin
    OUT_AR1H = A0 | B0;
    OUT_AR2H = A1 | B1;
    OUT_AR3H = A2 | B2;
    OUT_AR4H = A3 | B3;
end

// Preparación para resta baja
reg y0, y1, y2, y3;
always @(*) begin
    y0 = A0 & B0;
    y1 = A1 & B1;
    y2 = A2 & B2;
```

Funcion #11



```
module operation_block_13 (
    input wire A0, A1, A2, A3, B0, B1, B2, B3,
    output wire OUT_AR1H, OUT_AR2H, OUT_AR3H, OUT_AR4H, OUT_AR5H,
    output wire OUT_AR1L, OUT_AR2L, OUT_AR3L, OUT_AR4L, OUT_AR5L,
    output reg OUT_LG1H, OUT_LG2H, OUT_LG3H, OUT_LG4H,
    output reg OUT_LG1L, OUT_LG2L, OUT_LG3L, OUT_LG4L
);

// Lógica AND-NOT alto
always @(*) begin
    OUT_LG1H = A0 & ~B0;
    OUT_LG2H = A1 & ~B1;
    OUT_LG3H = A2 & ~B2;
    OUT_LG4H = A3 & ~B3;
end

// Lógica OR-NOT bajo
always @(*) begin
    OUT_LG1L = A0 | ~B0;
    OUT_LG2L = A1 | ~B1;
    OUT_LG3L = A2 | ~B2;
    OUT_LG4L = A3 | ~B3;
end

reg y0, y1, y2, y3;
always @(*) begin
    y0 = A0 & B0;
    y1 = A1 & B1;
    y2 = A2 & B2;
    y3 = A3 & B3;
end

sumadora INST_ADD_H (
    .A3(y3), .A2(y2), .A1(y1), .A0(y0), .C_IN(1'b0),
    .B3(A3), .B2(A2), .B1(A1), .B0(A0),
    .S3(OUT_AR4H), .S2(OUT_AR3H), .S1(OUT_AR2H), .S0(OUT_AR1H), .C_OUT(OUT_AR5H)
);
```

Funcion #13

Nota: Para la construccion de la restadora, sumadora se utilizaron los mismo diagramas logicos y codigo que se utilizaron en la tarea de la multiplicadora.

IV. **Conclusión**

Durante el desarrollo de este laboratorio se consiguió diseñar y simular una Unidad Aritmético-Lógica (ALU) de 4 bits empleando lógica combinacional. Se implementaron diversas funciones aritméticas y lógicas, seleccionadas tanto de forma manual como en función de la matrícula, lo que permitió integrar varias operaciones en un único bloque funcional. El uso de multiplexores facilitó la selección de las salidas, evidenciando el principio de control presente en una ALU real. Asimismo, el diseño fue llevado a Verilog con el objetivo de posibilitar su implementación en hardware, acercando el proyecto al uso de FPGA y sistemas digitales reales. Esta experiencia permitió fortalecer los conocimientos en diseño digital, estructuración de código y comprender la importancia de las ALU dentro de los sistemas computacionales.